

mongos instances cache the metadata and use it to route read and write operations to the correct shards. Also called as query router, *mongos* updates the cache when there are metadata changes for the cluster, such as chunk splits or addition of a shard. A sharded cluster can contain more than one query router to divide the client request load.

MongoDB storage engine

The storage engine is the component of the database that is responsible for managing how data is stored, both in memory and on disk. MongoDB supports multiple storage engines, as different engines perform better for specific workloads. The engine can be selected based on the need of the application. There are three engines.

WiredTiger: The WiredTiger storage engine is the default storage engine starting in MongoDB version 3.2. It provides a document-level concurrency model, allowing multiple users to write on the same collection in different documents at the same time.

Encrypted storage engine: The data is encrypted at rest by using this engine. A decryption key is needed to read the data. This engine is available in the MongoDB Enterprise edition.

In-memory storage engine: All the data, including the configurations, are stored in the memory. This engine provides quicker and low latency by avoiding I/O from disks.

However, data is not persistent.

The MongoDB data storage hierarchy is:

- Clusters
 - Database - <Collection of Collections>
 - Collections - <Similar to RDBMS Table>
 - Document - <JSON single record> <Single Row>
 - Key and value pair

Document

A document is a way to organise and store data as a set of field-value pairs. A field is a unique identifier for a datapoint, while value is data related to a given identifier. The documents are viewed in the JSON (JavaScript Standard Object Notation) format.

A JSON object is defined in the following format:

- Starts and ends with curly braces { }
- Separates each key and value with a colon :
- Separates each key:value set with a comma ,
- Keys must be written within double quotes ""
- It is represented based on the data type of value
- Value can be a number, string, objects (sub-document)

However, JSON has a few limitations. As it's text based, it consumes more space. It supports limited data types and only the UTF-8 format.

Hence, documents are stored in the BSON format. BSON stands for Binary JSON, and addresses the limitations of JSON by storing the data in the binary format. Optimised for better speed, limited space usage and high performance, it supports

multiple data types. BSON is faster to parse and lighter to store than JSON. Its drawback is that only machines can read it.

Collections

Documents are stored in collections. There can be many collections per database and many documents per collection.

- *Collection:*
 - Should not connect \0, . and \$
 - Should not start with a system
- *SubCollection:*
 - Syntactic sugar
 - Holds sub-collections

Database

Since MongoDB is a NoSQL database, the data is not saved in rows or columns but in documents as described earlier. Here is the folder structure.

- *admin*
 - Authentication and authorisation
- *local*
 - Database stores data specific to a single server
 - Stores the data used in the replication process
 - Local itself is never replicated
- *config*
 - Stores information about each shard

Atlas, a MongoDB cloud

Atlas is one of the MongoDB clouds, and creates three replicas.

Replica set: A few connected machines store the same data to ensure that if something happens to one of the machines the data remains intact.

Instance: This is a single machine locally or in the cloud, running a certain software. In our case, it is the MongoDB database.

Cluster: A group of servers that store your data.

The **MongoDB client** is a tool, library, or driver that allows applications to connect to and interact with a MongoDB database. It provides the functionality needed to perform database operations, like connecting to a MongoDB instance, querying documents, updating records, deleting data, and managing collections and databases.

Written in JavaScript, **Mongo Shell** accepts all the JavaScript functions. Here are some example commands:

```
> help
> db.listingsAndReviews.updateOne.help
db.collection.updateOne(filter, update, options):
```

It updates a single document within the collection based on the filter.

```
> db.listingsAndReviews.updateOne
[Function: updateOne] AsyncFunction {
```